Frontend · Mundo Pokémon — Fase 1 · Estructura HTML

Ejercicio de cumplimiento obligatorio

Esta práctica es de **cumplimiento obligatorio**, ya que se utilizará como referencia de evaluación continua y servirá de base para las fases posteriores de CSS, JavaScript y React.

Propósito de esta fase

En esta primera fase vas a construir la **estructura HTML** de una interfaz inspirada en una Pokédex.

Aquí no toca todavía centrarse en el acabado visual final ni en la interacción con JavaScript. El objetivo principal es que sepas:

- analizar una interfaz;
- identificar sus bloques;
- traducirla a HTML con sentido;
- y dejar una base sólida para las siguientes fases de CSS, JavaScript y React.

Contexto de la práctica transversal

Este ejercicio forma parte de una práctica transversal del bloque de frontend.
La misma interfaz evolucionará más adelante en:

- **Fase 2 · CSS**
- **Fase 3 · JavaScript**
- **Fase 4 · React**

Qué se va a trabajar aquí

En esta fase se evalúa sobre todo:

- la estructura del documento;
- la semántica general;
- la organización del contenido;

- la repetición coherente de tarjetas;
- y tu capacidad para pasar de una referencia visual a un HTML bien planteado.

---

1. Competencias que vas a trabajar

Al terminar esta fase deberías ser capaz de:

- crear un documento HTML completo y correcto;
- identificar zonas principales de una interfaz;
- estructurar una página con sentido semántico;
- construir bloques repetibles de contenido;
- preparar un HTML que luego pueda maquetarse bien con CSS;
- y dejar una base suficientemente clara para añadir interacción después.

---

2. Organización del trabajo

2.1. Carpeta de trabajo

Resuelve esta fase dentro de la siguiente carpeta del repositorio:

```
frontend/
└── mundo-pokemon/
    └── index.html
```

2.2. Recomendación de rama

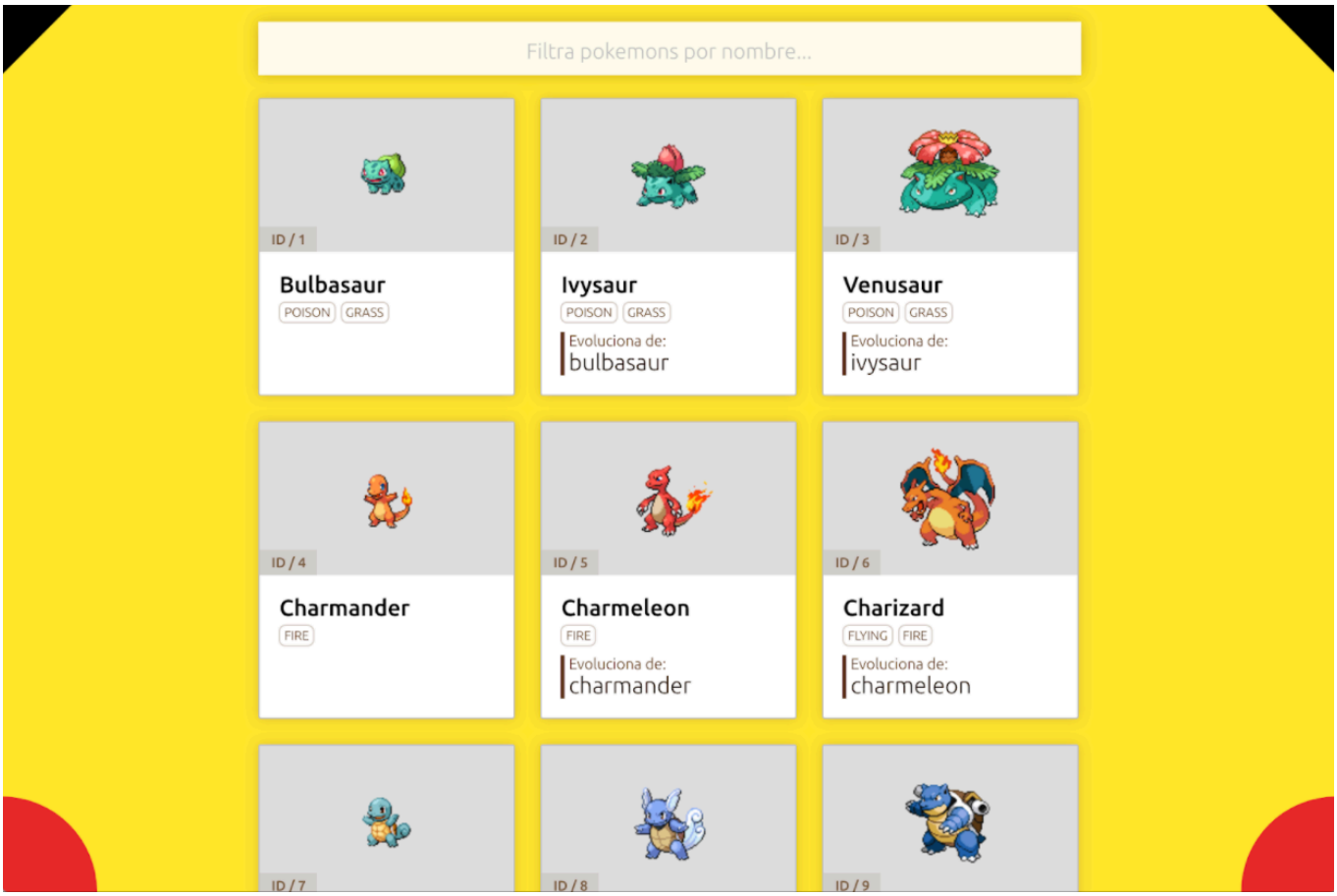
Si vas a trabajar con ramas, usa una rama específica para esta fase.

Ejemplo recomendado:

```
feature/frontend-mundo-pokemon-phase-01-html
```

---

3. Material de referencia y origen de las imágenes



3.1. Qué pokémon usar

Para esta fase usa **los nueve primeros pokémon** que aparecen en la referencia visual:

1. Bulbasaur
2. Ivysaur
3. Venusaur
4. Charmander
5. Charmeleon
6. Charizard
7. Squirtle
8. Wartortle
9. Blastoise

En esta primera fase, en la que vas a construir solo la estructura HTML, la interfaz todavía no se parecerá visualmente a la imagen proporcionada.

3.2 Información de los pokémon que vas a usar

```
1 | bulbasaur | grass, poison
2 | ivysaur | grass, poison | bulbasaur
```

```
3 | venusaur | grass, poison | ivysaur
4 | charmander | fire
5 | charmeleon | fire | charmander
6 | charizard | fire, flying | charmeleon
7 | squirtle | water
8 | wartortle | water | squirtle
9 | blastoise | water | wartortle
```

3.3 ¿De dónde saco las imágenes?

Las imágenes que se están usando en la interfaz de cada uno de ellos tienen una URL que siguen el siguiente patrón:

```
https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/[#ID]
.png
```

Por ejemplo, la imagen del primer pokémon puedes encontrarla en la siguiente url: https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1.png

---

4. Reglas generales de esta fase

En todos los ejercicios o tareas de esta fase:

- escribe el documento HTML completo;
- usa `lang="es"` en el documento;
- incluye `charset`, `viewport` y `title`;
- mantén una indentación limpia;
- usa nombres de clases razonables si decides añadirlas;
- no te obsesiones todavía con el diseño visual final;
- no resuelvas esta fase con tablas;
- no conviertas el HTML en una acumulación de `div` sin criterio.

> ⚠ **Error frecuente**
>
> En una interfaz visualmente atractiva es fácil querer empezar por “cómo se ve”.
> En esta fase el orden correcto es:
>
> 1. entender qué bloques existen;
> 2. estructurarlos en HTML;

3. y solo después pensar en la siguiente fase de CSS.

---

5. Ejercicio

Construye el marcado HTML necesario para representar de forma coherente la interfaz de referencia y dejarla preparada para su maquetación en CSS en sesiones posteriores.

Construye el archivo `index.html` completo con:

- `DOCTYPE`;
- `html` con `lang="es"`;
- `head` con:
  - `meta charset="UTF-8"`,
  - `meta name="viewport"`,
  - `title`;
- `body` con una estructura principal clara.

Qué debes observar

No pienses solo “veo tarjetas”. Observa bien la imagen y reflexiona:

- ¿hay cabecera? ¿qué hace de cabecera?;
- ¿qué puede ser la caja que hay encima del listado?;
- ¿qué se repite?;
- ¿qué información tiene jerarquía dentro de cada tarjeta?.

Importante

En esta fase no se evalúa que la interfaz se parezca visualmente a la imagen de referencia. Se evalúa que sepas traducirla a una estructura HTML lógica y reutilizable.

Solo vas a implementar el marcado HTML, así que conviene pensarlo bien para que en sesiones posteriores puedas maquetarlo con CSS más fácilmente.

En lo primero que nos hemos fijado: La tarjeta Pokémon

Cuidado con este marcado HTML. Si esta tarjeta está mal planteada, luego repetirás un patrón deficiente nueve veces.
Conviene dedicarle tiempo a pensar.

- ¿Qué etiqueta HTML representa mejor la tarjeta completa?
- ¿Hay secciones dentro de la tarjeta?
- ¿Hay algún texto que tenga jerarquía sobre otros?
- ¿Qué elementos son más importantes y cuáles secundarios?
- ¿Las nueve tarjetas siguen el mismo patrón estructural? ¿Hay elementos que están en unas tarjetas y no en otras?

Buen criterio

No siempre existe una única solución correcta. Lo importante es que tu decisión sea razonable y defendible.

---

6. Cierre

El ejercicio se considerará bien resuelto si:

- el documento HTML está completo;
- la estructura general se entiende sin CSS;
- la zona de búsqueda está bien planteada;
- las tarjetas siguen un patrón coherente;
- la semántica elegida es razonable;
- y el HTML queda preparado para la fase de CSS.

> Al terminar esta fase deberías haber construido una base HTML clara, semántica y repetible, lista para maquetar en CSS sin tener que rehacer la estructura.

Idea final

Una interfaz sólida empieza por un HTML sólido.
Si construyes bien la base, todo lo que venga después —CSS, JavaScript y React— será más fácil, más limpio y más mantenible.